

DEEP LEARNING SCHOOL 2025

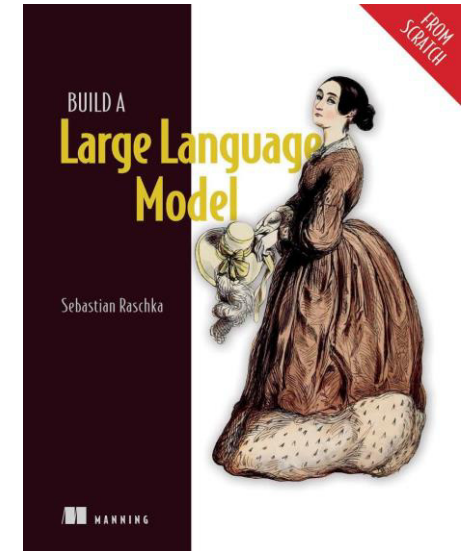
From June 23 to July 4 · Campus SophiaTech, Sophia Antipolis

7th edition

Tutorials: Build your own LLM from Scratch

Frederic Precioso, Diane Lingrand, Celia D'Cruz, Remy Sun, Lucile Sassatelli, Michel Riveill, Xingyuan Kang

- Inspired and partly based on “Build a large language model (from Scratch)” by Sebastian Raschka
- A consistent program to overview the whole process of designing solutions such as ChatGPT, DeepSeek, LeChat, etc, AND to prepare you for the conferences of the Deep Learning School
 - ~1 hour lecture on main principles
 - ~2 hours Lab in Python with Google Colab
- A central objective to intend to provide you with a critical understanding of LLMs



*Recommended reading,
Sebastian Raschka*

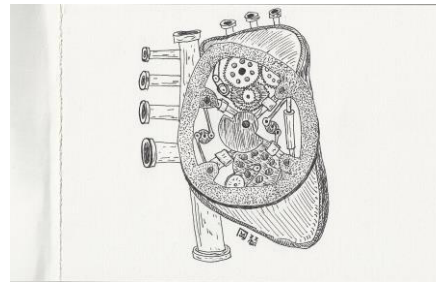
Tutorials

1. Tokenization
2. Introduction to Pytorch and classification using Neural Networks
3. Transformers, Encoder, Self-Attention, positional encoding, Text classification
4. Transformers, Decoder, Masked-Attention, Next word generation
5. Learn to answer instructions (Instruct)
6. Vision Transformers, Visual Language Models, MultiModal Language Models
7. *Retrieval-Augmented Generation (RAG) & Prompt Engineering*
8. *Reinforcement Learning with Human Feedback (RLHF), DPO*

A bit of “History” to start...

We want to believe... and have for a long time...

Artificial intelligence, as a field of research, is considered to have been created at the conference held on the campus of **Dartmouth College** in the summer of **1956**, even though this notion has been present since antiquity: Hephaestus built metal automatons to work for himself or protect others (Talos protecting the goddess Europa) and created Pandora, the first human woman in Greek mythology, from clay; the Golem, in Jewish folklore, is also made of clay, Yan Shi built humanoid automatons in the 10th century BC. during the Zhou Dynasty.



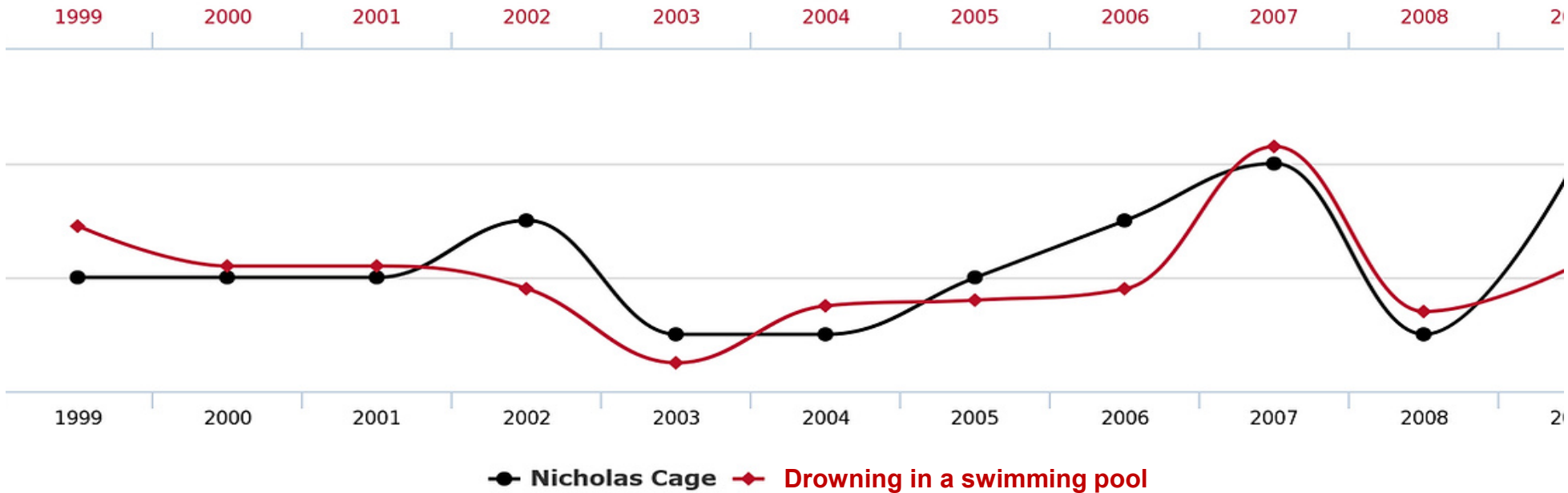
*Maria dans
Metropolis 1927
De Fritz Lang*

Sources :

- Wikipedia, <https://www.greeklegendsandmyths.com/automatons.html>
- http://www.alanturing.net/turing_archive/pages/Reference%20Articles/what_is_AI/What%20is%20AI02.html
- Stanford Encyclopedia of Philosophy: <https://plato.stanford.edu/entries/artificial-intelligence/>

We confuse too easily correlation and causality

Number of people who drowned by falling into a pool
correlates with
Films Nicolas Cage appeared in

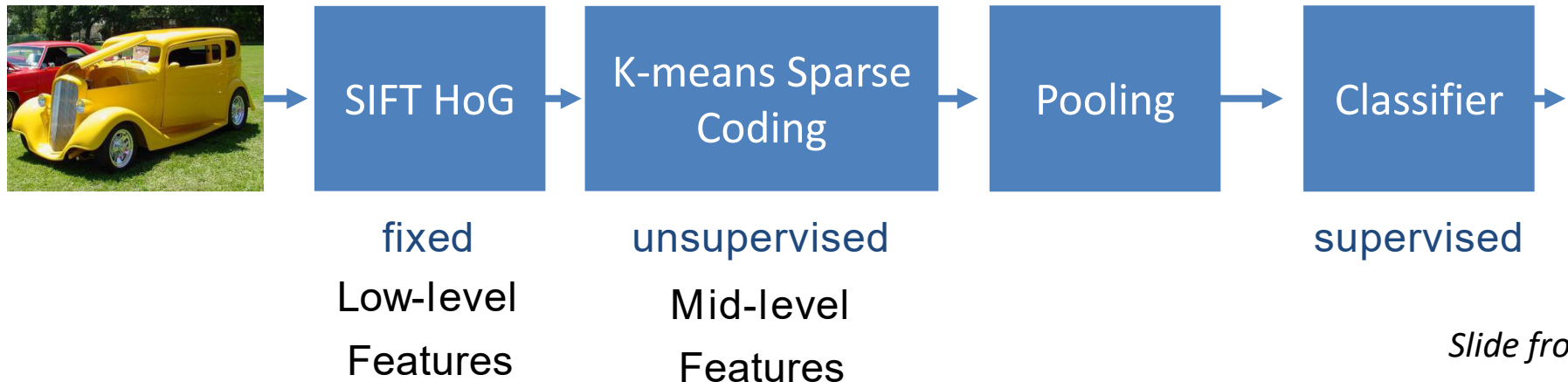


Architecture of “mainstream” pattern recognition systems

- Modern architecture for pattern recognition
 - Speech recognition: early 90's – 2011



- Object Recognition: 2006 - 2012

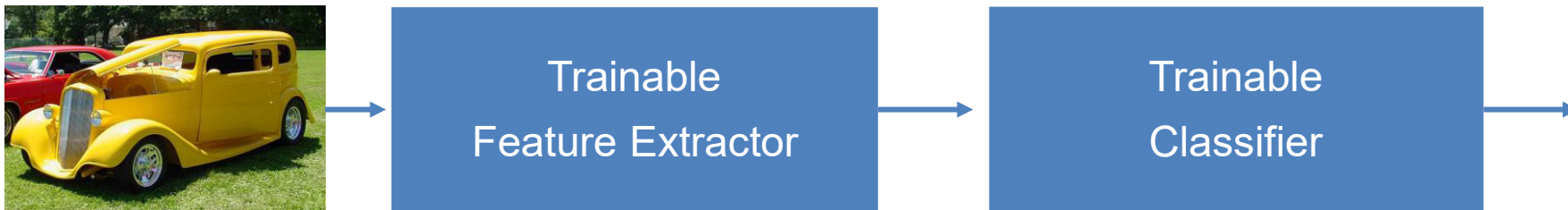


Deep learning = Learning representations/features

- The traditional model of pattern recognition (since the late 50's)
 - Fixed/engineered features (or fixed kernel) + trainable classifier



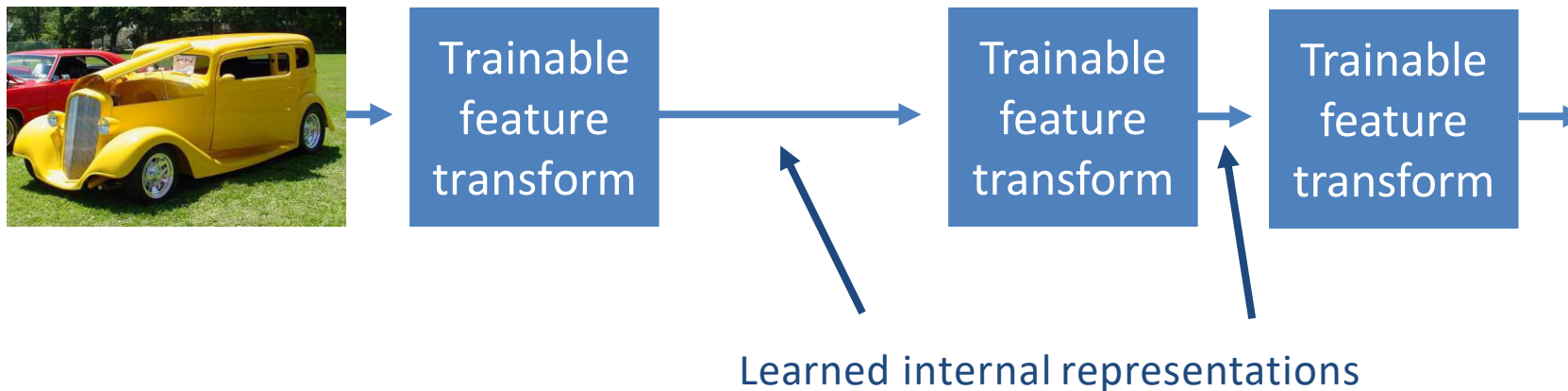
- End-to-end learning / Feature learning / Deep learning
 - Trainable features (or kernel) + trainable classifier



Slide from Yann LeCun

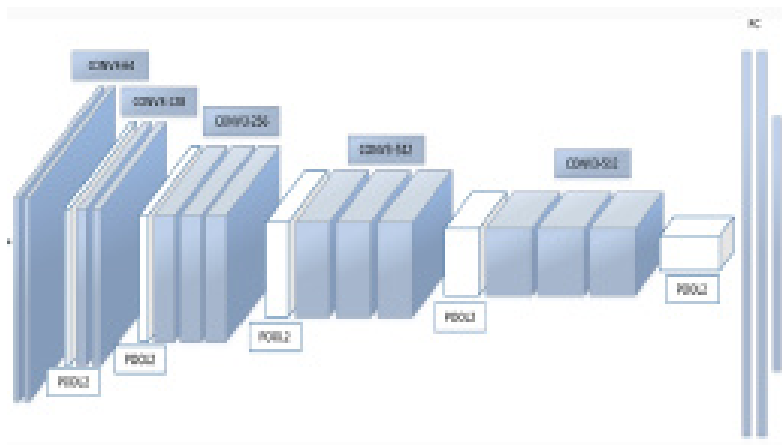
Trainable feature hierarchies: end-to-end learning

- A hierarchy of trainable feature transforms
 - Each module transforms its input representation into a higher-level one.
 - High-level features are more global and more invariant
 - Low-level features are shared among categories

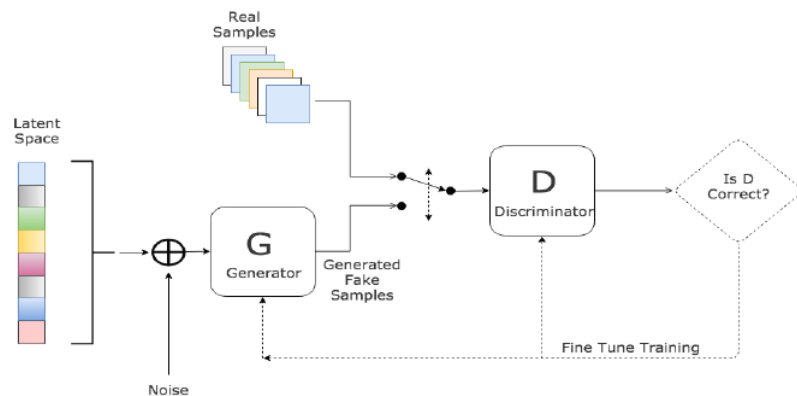


- How can we make all the modules trainable and get them to learn appropriate representations?

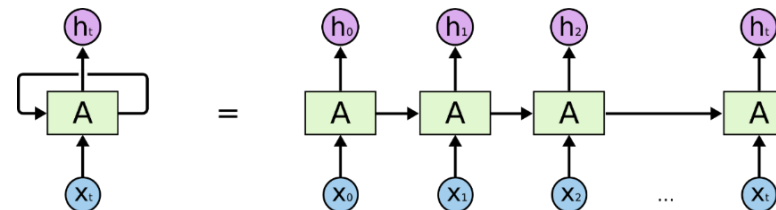
Network architectures



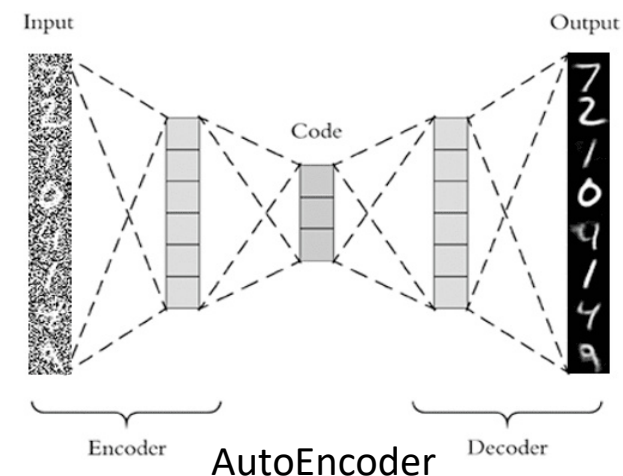
Convolutional Neural Networks / Spatial Correlation
(images, sounds...)



Generative Adversarial Network
(Data Generator, Domain adaptation...)

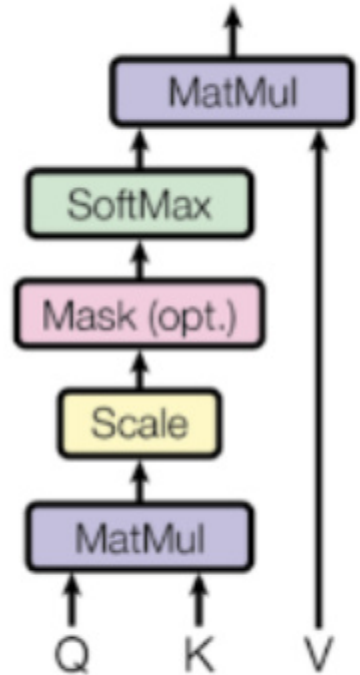


Recurrent Neural Network / Sequential Correlation
(machine translation, speech 2 text, time series...)

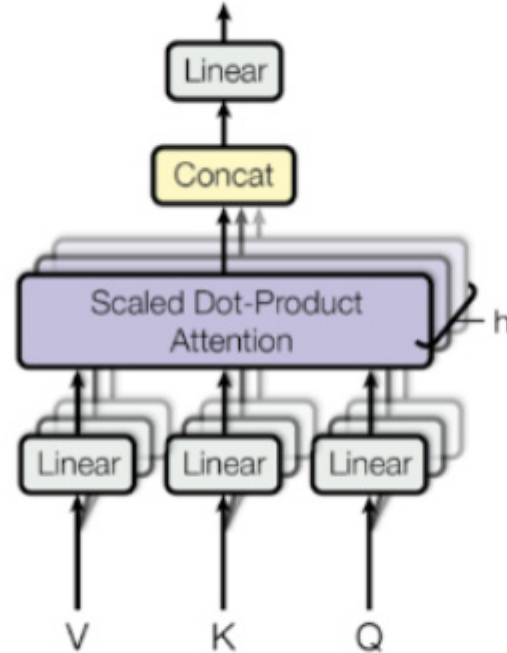


AutoEncoder
(Dimension reduction, compression,
denoising/data cleansing, data generator...)

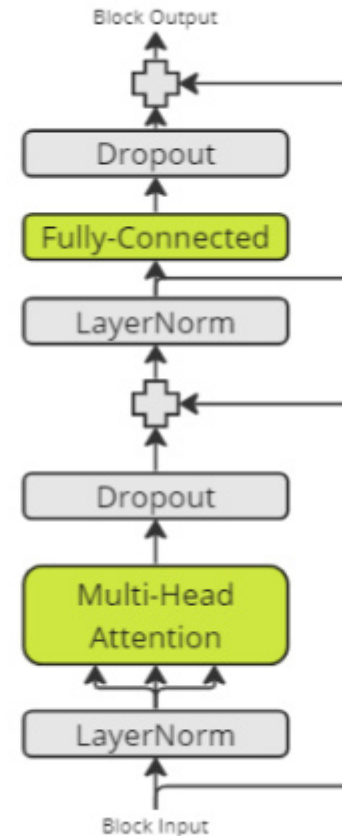
Among others...Transformers



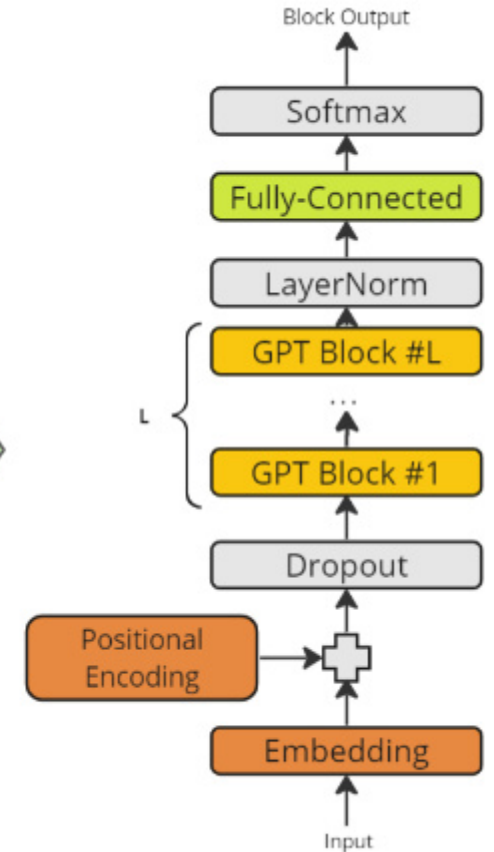
Scaled Dot-Product Attention



Multi-Head Attention



Transformer Block
(GPT Block)



GPT Model

DEEP LEARNING SCHOOL 2025

From June 23 to July 4 · Campus SophiaTech, Sophia Antipolis

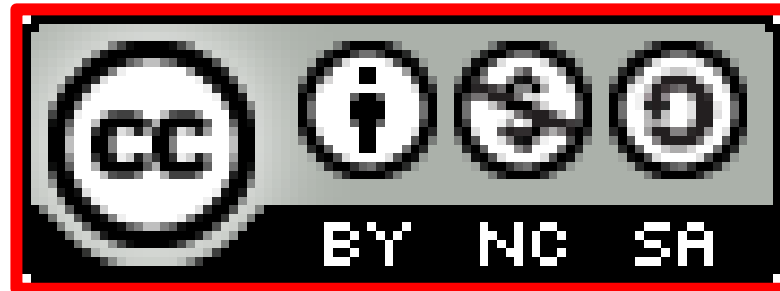
7th edition

Tutorial I: Tokenization

Frederic Precioso, Diane Lingrand, Michel Riveill,
Celia D'Cruz, Lucile Sassatelli

This content is under License: CC BY-NC-SA

- Deep Learning School by EFELIA Côte d'Azur under Licence [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-nc-sa/4.0/).



Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)

This is a human-readable summary of (and not a substitute for) the [license](#). [Disclaimer](#).

You are free to:

Share — copy and redistribute the material in any medium or format

Adapt — remix, transform, and build upon the material

[Share your work](#)

[Use & remix](#)

[What We](#)

Under the following terms:



Attribution — You must give [appropriate credit](#), provide a link to the license, and [indicate if changes were made](#). You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.



NonCommercial — You may not use the material for [commercial purposes](#).



ShareAlike — If you remix, transform, or build upon the material, you must distribute your contributions under the [same license](#) as the original.

No additional restrictions — You may not apply legal terms or [technological measures](#) that legally restrict others from doing anything the license permits.

Disclaimer & Foreword

If any content in this presentation is yours but is not correctly referenced or if it should be removed, please contact us and we will correct it.

We deliberately took some shortcuts and simplified or ignored some technical/algorithmic aspects to focus on the core principles and the key mechanisms of LLMs.

- Construction d'un Large Language Model (LLM)
 - Préparation des données
 - Mécanismes d'attention
 - Architecture LLM (Transformers)
- Modèles fondation
- Développement d'applications
- Extension à d'autres modalités (par exemples Images, Vidéos, Sons...)
- Questions éthiques autour de ces modèles

Plan du Cours

- Construction d'un Large Language Model (LLM)
 - Préparation des données
 - Tokenisation
 - Identifiant numérique par token
 - Evaluation des schémas de tokenisation
 - Glitch Tokens
 - Embedding
 - Equité des schémas d'embedding
 - Mécanismes d'attention
 - Architecture LLM (Transformers)
- Modèles fondation
- Développement d'applications
- Extension à d'autres modalités (par exemples Images, Vidéos, Sons...)
- Questions éthiques autour de ces modèles

DATA PREPARATION

Context

Tokenization is at the heart of most of the weird behaviors of LLMs.

What makes that:

- an LLM cannot spell words? Tokenization
- an LLM cannot perform very simple word processing tasks like inverting a string? Tokenization
- an LLM is worse in languages other than English (e.g. Japanese)? Tokenization.
- an LLM is bad with simple arithmetic? Tokenization
- GPT-2 had more trouble than necessary to code in Python? Tokenization
- an LLM stops abruptly when it sees the “<|endoftext|>”? Tokenization
- I am getting a strange warning about an "end space"? Tokenization
- an LLM stops if I question it about "*SolidGoldMagikarp*"? Tokenization
- I should prefer to use YAML rather than JSON with an LLM? Tokenization
- An LLM is not really an end-to-end language model? Tokenization

What is the real source of the difficulties of LLMs? Tokenization!!

- We will call **token**, a portion of input data that will represent a "unitary" information entity.
- How do you determine the right portion of the data?
- For textual data, the answer depends, for example, on the language or on the context.

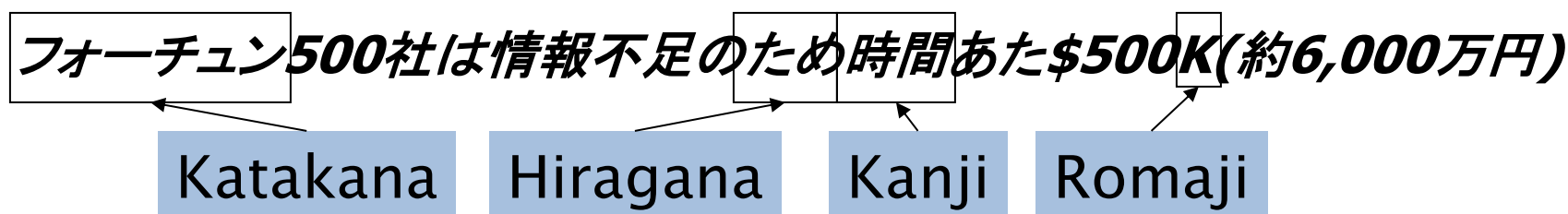
Issues in Tokenization

- *Finland's capital → Finland AND s? Finlands? Finland's?*
- *Hewlett-Packard → Hewlett and Packard as two tokens?*
 - *state-of-the-art: break up hyphenated sequence.*
 - *But then: co-education? lowercase, lower-case, lower case?*
 - *It can be effective to get the user to put in possible hyphens*
- *San Francisco: one token or two?*
 - *How do you decide it is one token?*
- *3/20/91; Mar. 12, 1991; 20/3/91? 55 B.C.? (800) 234-2333?*

Language issues in Tokenization

- German noun compounds are not segmented
Lebensversicherungsgesellschaftsangestellter → ‘life insurance company employee’
 — German retrieval systems benefit greatly from a compound splitter module Can give a 15% performance boost for German
- Chinese and Japanese have no spaces between words:
 莎拉波娃现在居住在美国东南部的佛罗里达。
- Further complicated in Japanese, with multiple alphabets intermingled.
 Dates/amounts in multiple formats:

フォーチュン500社は情報不足のため時間あた\$500K(約6,000万円)



Katakana Hiragana Kanji Romaji

- End-user can express query entirely in hiragana!

Language issues in Tokenization

- Arabic (or Hebrew) is basically written right to left, but with certain items like numbers written left to right
- Words are separated, but letter forms within a word form complex ligatures:

استقلت الجزائر في سنة 1962 بعد 132 عام من الاحتلال الفرنسي.
 ← Start ← → ← →

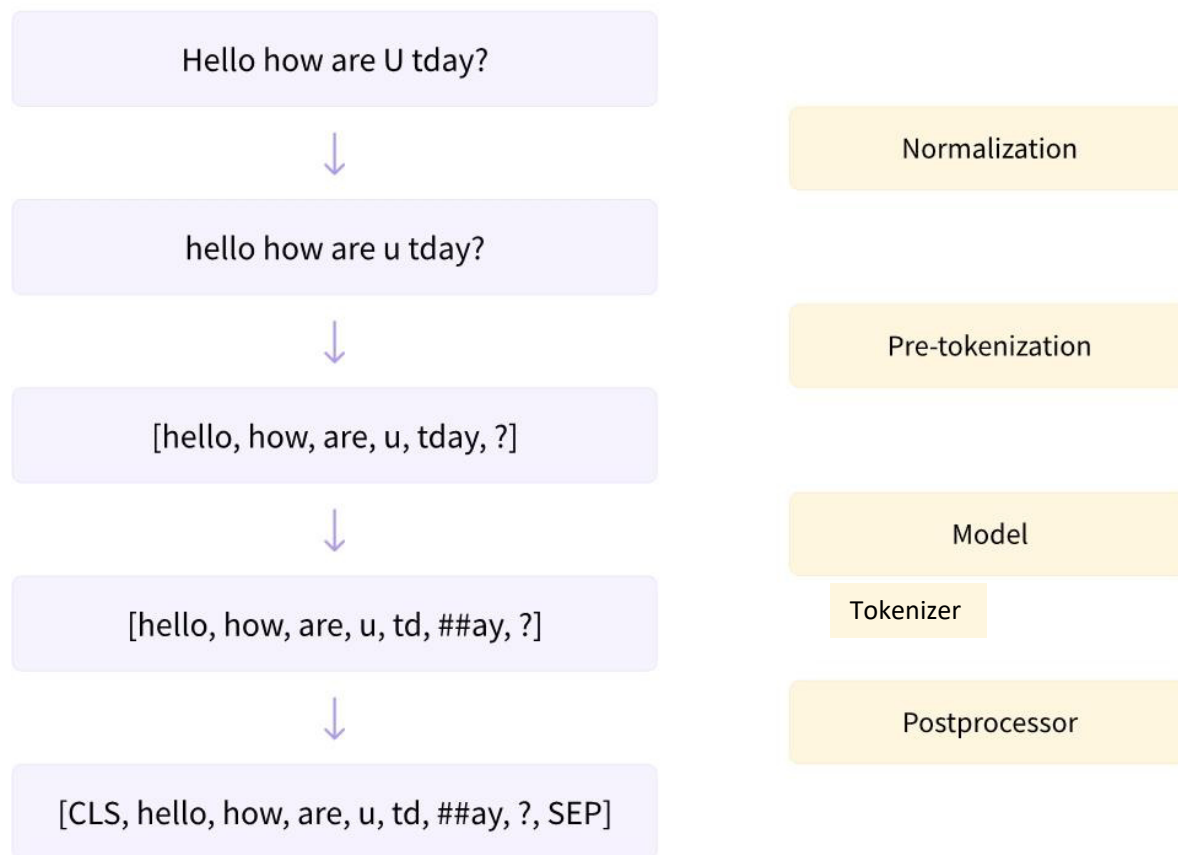
'Algeria achieved its independence in 1962 after 132 years of French occupation.'

- **From this slide, we will explain the principles of tokenization for languages whose different words are separated by spaces.**

Tokenization

Preprocessing

- Before focusing on the models to represent a text, two steps of data pre-processing are generally applied:
 - Normalization
 - Pre-tokenization



Normalization

- The normalization step involves a general cleanup, such as removing unnecessary spaces, lowercase and/or removing accents.
- Normalization can lead to losing valuable and/or important information: If we consider the words *Turkey* and *turkey* for example. The first is the country. The second is a bird... Two totally different things. Same things with names of persons or locations...
- This simple example shows us how much meaning or information is carried by a capital letter alone.

- Stemming:
 - Reducing terms to their root before indexing them
 - The term "stemming" suggests a rough cut of the affix. Moreover, it is not necessary for a stem to be a correctly spelled word.
 - This step obviously depends very strongly on the language considered.
 - For example, *automata*, *automatic*, *automation* are all reduced to *automat*.
- Stemming can also be considered a form of dimensionality reduction... A reduction in the size of our vocabulary at the expense of a possible loss of information: For example, *compressed* and *compression* are both replaced by *compress*.

Lemmatization

- Lemmatization is another normalization technique that is more extensive down to the semantic root of a word – its lemma.
- Using a knowledge base of synonyms and word endings, a lemmatizer tries to consolidate only words that mean similar things into a single token.
- Reduction of inflectional variants to the basic words:
 - am, are, is → be
 - car, cars, car's, cars' → car
 - the boy's cars are different colors → the boy car be different color
- Lemmatization involves making an "appropriate" reduction to the form of a dictionary keyword
- Lemmatization is potentially more accurate than stemming but is also a form of dimensionality reduction... A reduction in the size of our vocabulary at the expense of a possible loss of information

- Each approach offers some advantages by reducing the size of the vocabulary, which allows for better generalization while simultaneously decreasing the computational effort.
- However, this comes at the cost of loss of information and reduced accuracy.
- **If it is not specifically useful for a certain application, standardization should be avoided. This is especially true if, for example, capital letters play an important role that we cannot afford to lose.**

Pre-tokenization

- As we will see in the following sections, a tokenizer cannot be trained on plain text alone.
- Instead, we first need to break the texts down into small entities, like words. This is where the pre-tokenization stage comes in.
- What is the best way to split a text to get a list of tokens?
Using first, Python's regular expression library `re`.

Example

- Using a simple text example, we can use the `re.split` command with the following syntax to split text into space characters:

```
import re
text = "Hello, world. This, is a test."
result = re.split(r'(\s)', text)
print(result)
```

- The result is a list of individual words, spaces, and punctuation characters:

```
['Hello,', ' ', ' ', 'world.', ' ', 'This,', ' ', ' ', 'is', ' ', ' ', 'a', ' ', ' ', 'test.']
```

- However, some words are still linked to punctuation characters that we want to have as separate entries in the vocabulary.
- We do not standardize all lowercase words so that our LLM can "learn" how to generate words with capital letters in a meaningful way.

Example

- Let's change the regular expression divisions on spaces (`\s`), commas, and periods (`[, .]`):

```
result = re.split(r'([, .]|\s)', text)
print(result)
```

- We can see that words and punctuation characters are now separate entries in vocabulary, just as we intended:

```
['Hello', ',', "'", ' ', 'world', '.', "'", ' ', 'This', ',', "'", ' ', 'is', ' ', 'a', ' ', 'test', '.', '']
```

- One small remaining problem is that the list always includes space characters. Optionally, we can safely remove these redundant characters as follows:

```
result = [item for item in result if item.strip()]
print(result)
```

- The resulting output without space characters looks like this:

```
['Hello', ',', "'", 'world', '.', 'This', ',', "'", 'is', 'a', 'test', '.']
```

Impact of Pre-Tokenization

- When developing a simple tokenization scheme, whether to encode the spaces in separate characters or remove them **depends on our application and its requirements**.
- Removing white space reduces memory and compute requirements. However, preserving spaces can be useful if we train models that are sensitive to the exact structure of the text (**e.g., Python code, which is sensitive to indentation and spacing**).
- For example, if we use GPT-2's pre-tokenizer (Tokenizer = BPE), it will separate on spaces and punctuation but keep the spaces and replace them with a `Ġ` symbol, which will allow it to recover the original spaces if we decode the tokens.

Back to the Lab

- Here, we remove spaces for simplicity and conciseness of tokenized outputs. Later, we will move to a tokenization scheme that includes spaces.
- The code can be modified to accommodate more punctuation characters, so that the following text:

```
text = "Hello, world. Is this-- a test?"  
result = re.split(r' ( ??? |\s) ', text)  
...
```

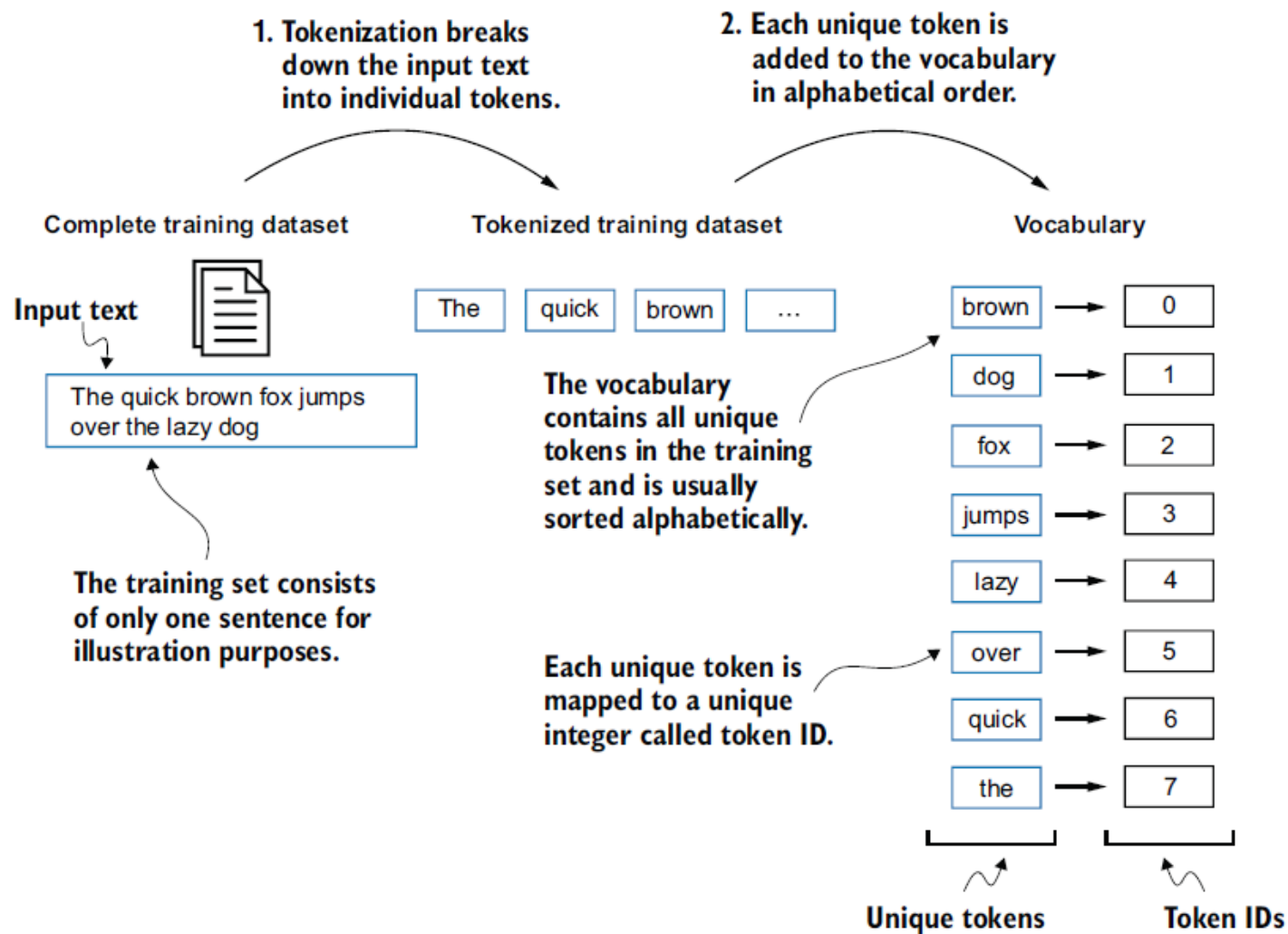
- Produces the result:

```
['Hello', ',', 'world', '.', 'Is', 'this', '--',  
'a', 'test', '?']
```

One unique ID per token

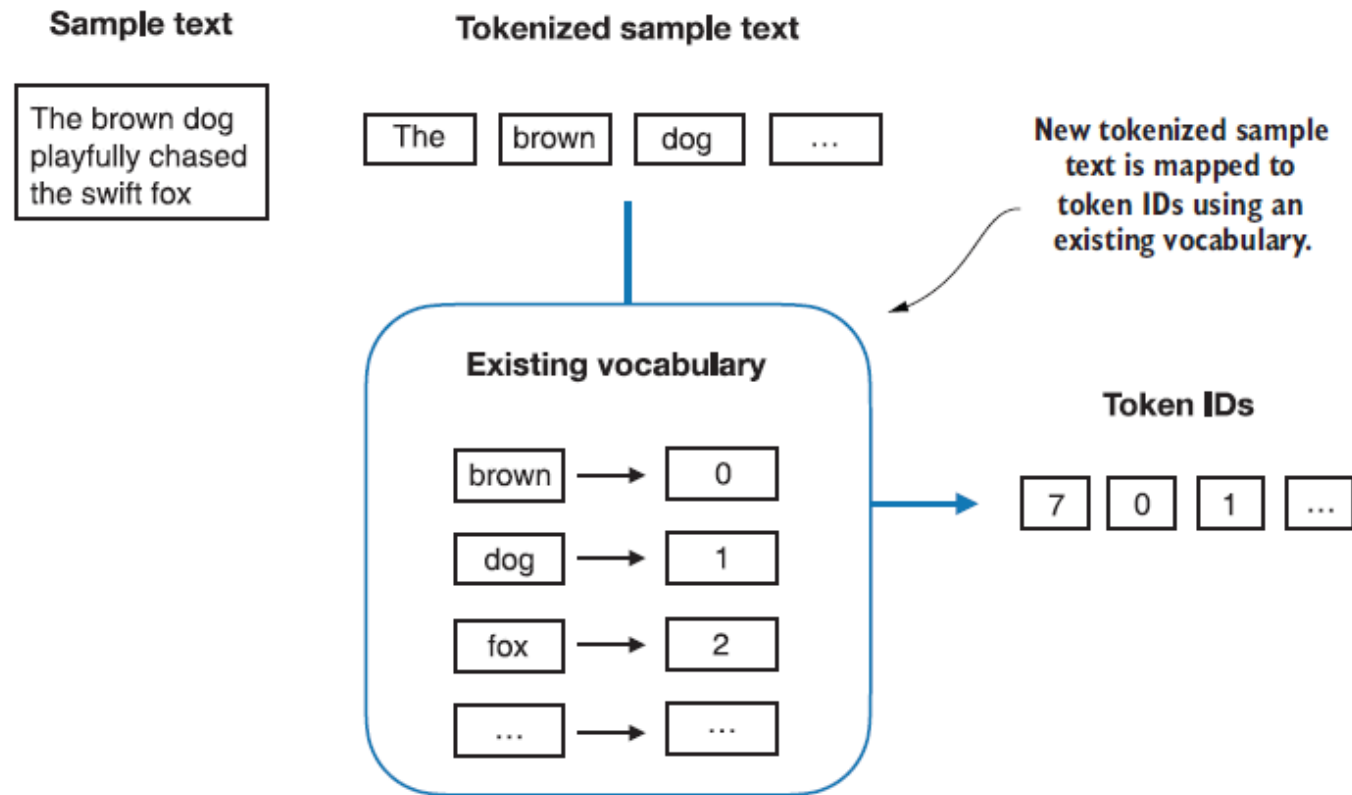
Identification

1. Next, we will convert these Python string tokens into integer representation to produce the token IDs.
2. To map tokens to IDs, we first need to create a vocabulary that defines how we associate each unique word and special character with a unique integer.

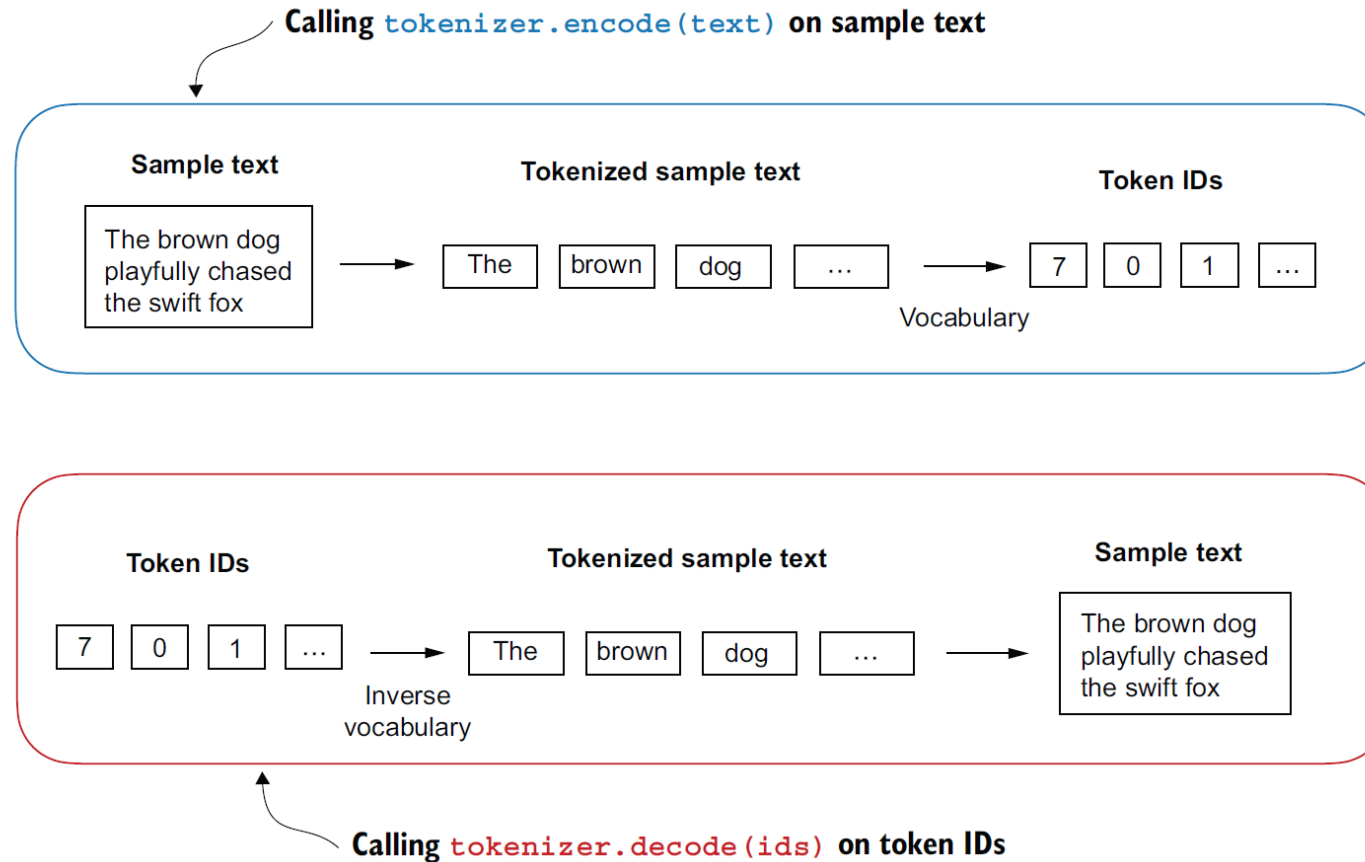


Vocabulary

- The goal now is to be able to use this vocabulary to encode any new text:



- Implementing your first Tokenizer to encode text but also of course decode token IDs, so that our LLM can generate text:



Out-Of-Vocabulary (OOV)

- What happens if we provide a new text with words that are not part of our vocabulary (they are called Out-of-Vocabulary or OOV)?

We get an error!

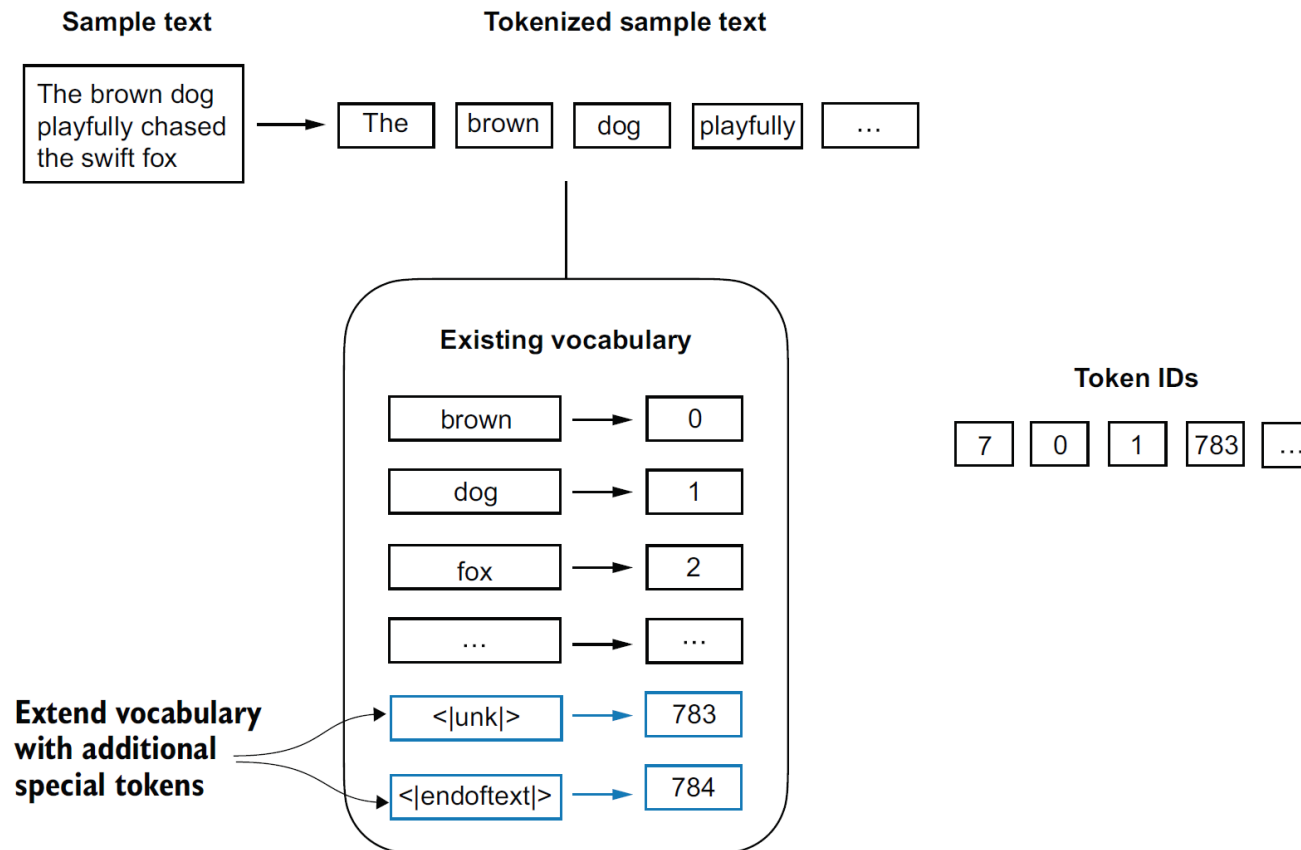
- How can we solve this problem?

Adding context tokens

- We need to change the tokenize to handle unknown words. We also need to address the use and addition of *special context tokens* that can improve the model's “understanding” of the context or other relevant information in the text.
- These special tokens can include markers for unknown words and document boundaries, for example.
- In particular, we are going to change the vocabulary and previous tokenization to support two new tokens, `<|unk|>` and `<|endoftext|>`.

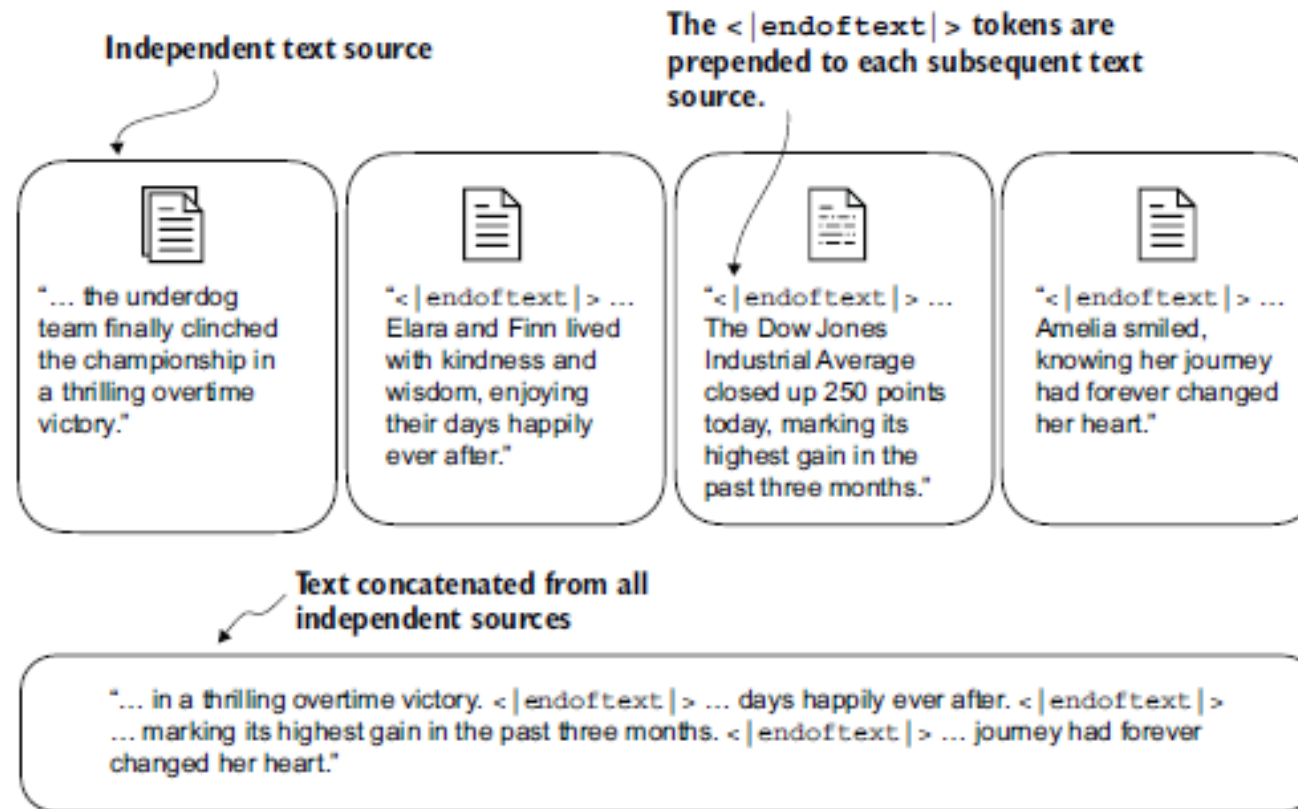
Adding context tokens

- We can modify the tokenizer to use a `<|unk|>` token, if the template encounters a word that is not part of the vocabulary.



Adding context tokens

- In addition, we add a token between unrelated texts.
- For example, when training GPT-like LLMs on multiple independent documents or books, it is common to insert a token before each document or book that follows a previous text source. This tells the algorithm that while these textual sources are concatenated for training, they are actually not related.



- Add these 2 specific tokens to the end of the previous vocabulary. You will be able to verify that they have the 2 highest IDs.
- Rewrite the Tokeniser to take into account these new tokens, and test your code by encoding text containing multiple sentences with words that are not all part of the vocabulary, and evaluate decoding.

Other Special Tokens

Depending on the LLM models, one can also consider additional special tokens such as the following:

- [BOS] (beginning of sequence)—This token marks the beginning of a text. It means for the LLM where content begins.
- [EOS] (end of sequence)—This token is positioned at the end of a text and is particularly useful when concatenating multiple unrelated texts, similar to `<|endoftext|>`.
- The tokenizer used for GPT models does not need any of these tokens; It only uses a `<|endoftext|>` token for simplicity.
- In addition, the tokenizer used for GPT models does not use either a `<|unk|>` token for non-vocabulary words. Instead, GPT models use a tokenizer based on the **Byte-Pair Encoding (BPE) algorithm**, which breaks down words into sub-units of words.

Tokenizer: Byte-Pair Encoding (BPE) algorithm

Token = Word? Character? Other?

If Token = Word

- Each unique space-delimited string is assigned a different integer
 - To limit vocabulary size, assign only the most frequent words to whole IDs
 - The other words are <unk>
- Pros and cons
 - Simple
 - Ability to compare/retrieve documents based on the number of tokens
 - Many words correspond to *unknown* (e.g. 1298, Bart's, Area-52, anachronism, ...)
 - Need for a large vocabulary
 - Does not model the similarity of related words like broke/broken or chair/chairs

Token = Word? Character? Other?

If Token = Character

- Each character is assigned to a unique integer
- Pros and cons
 - Simple
 - Every document in the alphabet can be fully modeled
 - Small vocabulary (< 100 integers needed for English)
 - Sometimes similar words will have similar sequences (broke/n)
 - The number of tokens is not a significant metric
 - The sequences of characters are long

Token = Word? Character? Other?

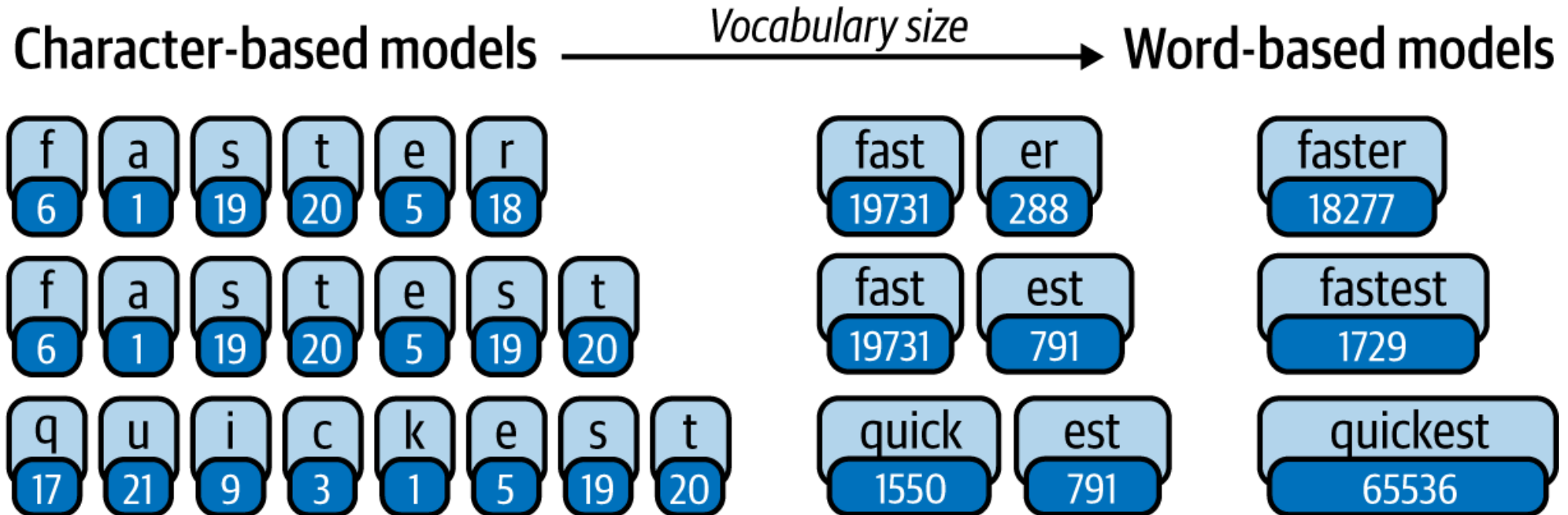
If Token = Word sub-unit

- The most common/frequent sequences of characters are assigned to unique integers
 - Rule 1: Do not break up frequent words into subunits
 - Rule 2: Break down infrequent/rare words into the most relevant sub-units possible
- Pros and cons
 - Every document in the alphabet can be fully modeled
 - The vocabulary size is flexible (e.g., 30K for BERT, 50K for GPT-3)
 - Sometimes similar words will have similar sequences (broke/n)
 - Need to find good tokenization of subwords

Token = Word? Character? Other?

	Character	Subword	Word
“Chair is broken”	c, h, a, i, r, ...	ch##, ##air, is, brok##, ##en	chair, is, broken
Vocabulary size	256	4K-50K	> 30K
Exhaustiveness	Perfect	Perfect	Incomplete
Independent Meaning	Bad	OK	Good
Token sequence length	Long	Average (e.g., 1.4 token per token)	A little shorter
Encodes the similarity of words	Lightly	A little better	Not at all

Token = Word? Character? Other?



Credits: Patel, A. A., & Arasanipalai, A. U. (2021). *Applied natural language processing in the enterprise*. "O'Reilly Media, Inc."

Token = Subword

- For example, if the word *boy* often appears in the vocabulary, it is not cut up. On the other hand, *boys* should be cut into *boy* and *s*.
- Splitting into subwords will help the model capture that different words with the same root token, such as for instance *tokens* and *tokenization* are (quite) similar.
- This will also help the model capture that the words *tokenization* and *modernization* are made of different roots but have the same suffix "*ization*" and are therefore used in the same syntactic situations.

Byte-Pair Encoding (BPE): Context

- We will now detail how one of the most used tokenization algorithms works in current LLMs: Byte-Pair Encoding (BPE)
- To do this, we will consider a corpus that uses these five words:

$\{ hug, pug, pun, bun, hugs \}$

- The basic vocabulary will then be $[b, g, h, n, p, s, u]$.
- If a piece of text that you tokenize, uses a character that is not in the training corpus, that character is converted to an unknown token.

Byte-Pair Encoding (BPE): Context

- We assume that words have the following frequencies in the corpus we consider:

(hug, 10), (pug, 5), (pun, 12), (bun, 4), (hugs, 5)

This means that "hug" is present 10 times in the corpus, "pug" 5 times, "pun" 12 times, "bun" 4 times and "hugs" 5 times.

Byte-Pair Encoding (BPE) algorithm

- BPE encoding is an algorithm, first described in 1994 by Philip Gage, that allows strings of text to be encoded in the form of arrays.
- The original version of the algorithm is focused on compression. It **replaces the most frequent byte pair, with a new byte that was not contained in the original dataset.**
- The BPE algorithm was used by OpenAI for the tokenization of the GPT model's pre-training. It is used by many transformers including GPT, GPT-2, Mistral, RoBERTa, BART, and DeBERTa.

BPE principles

- Suppose that the data to be encoded is:

aaabdaaabc

- The aa byte pair appears most often, so it will be replaced by a byte that is not used in the data, such as Z. There is now the following data and the replacement table:

ZabdZabc

Z=aa

BPE principles

- Then, the process is repeated with the byte pair ab , replacing it with Y :

$ZYdZYac$

$Y=ab$

$Z=aa$

- The only remaining literal byte pair only occurs once, and the encoding can stop here. Alternatively, the process could continue with recursive encoding of pairs of bytes, replacing ZY with X :

$XdXac$

$X=ZY$

$Y=ab$

$Z=aa$

Example in NLP

- The BPE algorithm will ensure that the most common/frequent words in the vocabulary are represented by a single token, while the rarer words will be split into two or more tokens.
- Let's go back to our example with a 36-word corpus with duplicates:

{hug: 10, pug: 5, pun: 12, bun: 4, hugs : 5}

- For this corpus, we obtain:
 - Tokenization per word: 5 tokens = *hug, pug, pun, bun, hugs*
 - Tokenization per Character: 7 tokens = *b, g, h, n, p, s, u*

- After getting this basic vocabulary, we add new tokens until the desired size of the vocabulary is reached by learning **merges**, i.e. rules for merging two pieces of the existing vocabulary to create a new one.
- So, at first, these merges will create two-character tokens, and then as the “training” progresses, longer subwords.
- At each stage of the tokenizer’s “training”, the BPE algorithm looks for the most frequent pair of existing tokens (by "pair" we mean two consecutive tokens in a word). This most common pair is the one that will be merged.
- This process is repeated.

- We start the training by dividing each word into characters (those that build our initial vocabulary) so that we see each word as a list of tokens:

$(h\ u\ g, 10), (p\ u\ g, 5), (p\ u\ n, 12), (b\ u\ n, 4), (h\ u\ g\ s, 5)$

- Then we go through all the lists of token pairs and collect how often they appear:
 - $(h\ u\ g, 10) \rightarrow (hu, 10), (ug, 10)$
 - $(p\ u\ g, 5) \rightarrow (pu, 5), (ug, 5)$
 - $(p\ u\ n, 12) \rightarrow (pu, 12), (un, 12)$
 - $(b\ u\ n, 4) \rightarrow (bu, 4), (un, 4)$
 - $(h\ u\ g\ s, 5) \rightarrow (hu, 5), (ug, 5), (gs, 5)$
- So we end up with the following suite of new tokens:
 $(pu, 17), (bu, 4), (un, 16), (hu, 15), (ug, 20), (gs, 5)$

- The most frequent pair is therefore (u, g) which is present in *hug*, *pug*, and *hugs*, for a total of 20 times in the vocabulary.
- So, the first merging rule learned by the tokenizer is $(u, g) \rightarrow ug$, which means that ***ug* is added to the vocabulary and the pair must be merged into all the words in the corpus.**
- At the end of this step, the vocabulary and corpus look like this:
 - Vocabulary: $[b, g, h, n, p, s, u, ug]$
 - Corpus: $(h\ ug, 10), (p\ ug, 5), (p\ u\ n, 12), (b\ u\ n, 4), (h\ ug\ s, 5)$

- We now have a few pairs that result in a token of more than two characters:

Corpus: (*h ug*, 10), (*p ug*, 5), (*p u n*, 12), (*b u n*, 4), (*h ug s*, 5)

- For example, the pair (*h*, *ug*) present 15 times in the corpus. The most common pair at this stage is (*u*, *n*), present 16 times in the corpus, so the second merging rule learned is $(u, n) \rightarrow un$. By adding this to the vocabulary and merging all the existing occurrences, we get:
 - Vocabulary: [*b*, *g*, *h*, *n*, *p*, *s*, *u*, *ug*, *un*]
 - Corpus: (*h ug*, 10), (*p ug*, 5), (*p un*, 12), (*b un*, 4), (*h ug s*, 5)

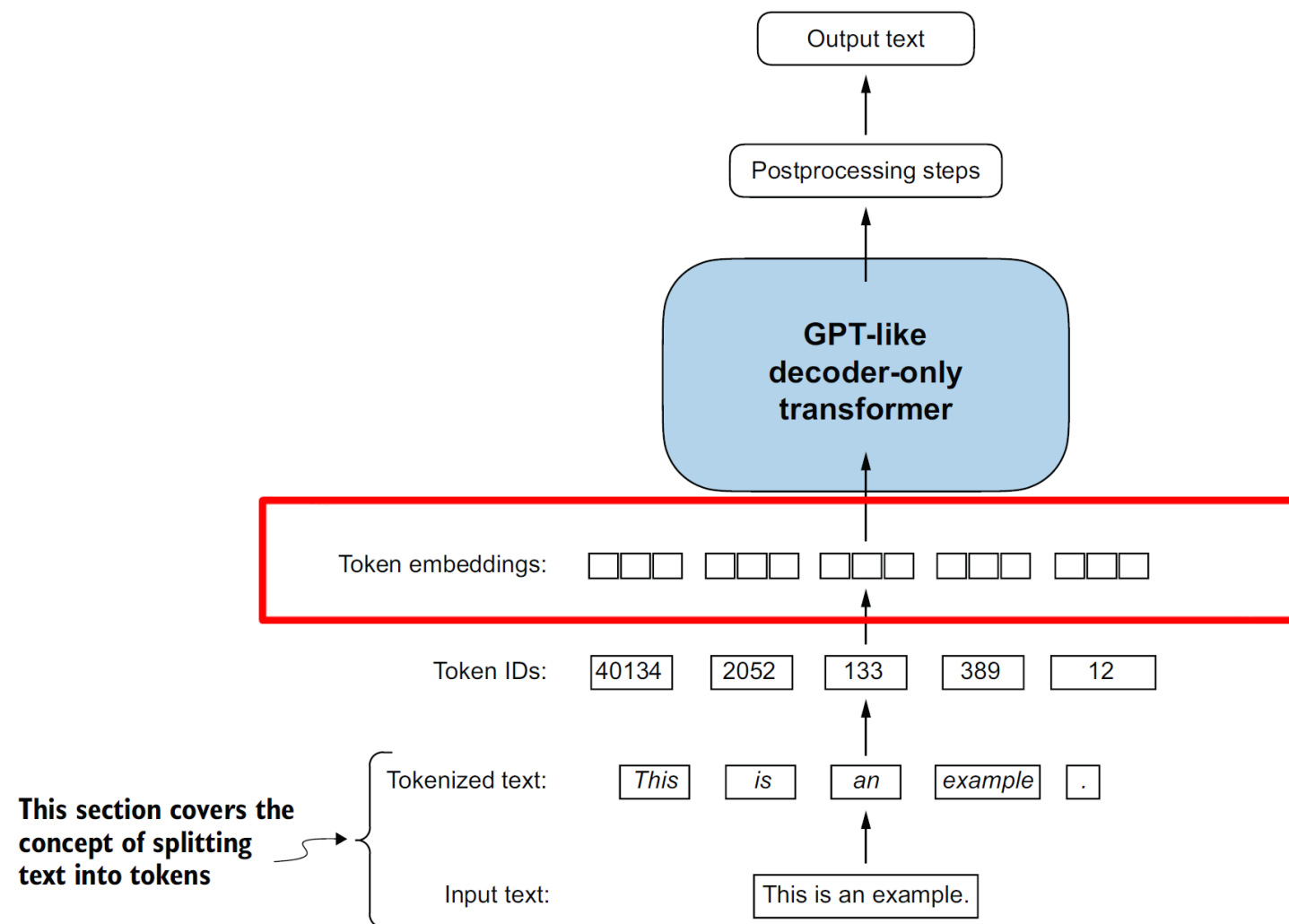
Algorithm

- Corpus: $(h\ ug, 10), (p\ ug, 5), (p\ un, 12), (b\ un, 4), (h\ ug\ s, 5)$
- Now the most common pair is (h, ug) so we learn the merging rule $(h, ug) \rightarrow hug$. So this gives us our first three-letter token. After the merge, the corpus looks like this:
 - Vocabulary: $[b, g, h, n, p, s, u, ug, un, hug]$
 - Corpus: $(hug, 10), (p\ ug, 5), (p\ un, 12), (b\ un, 4), (hug\ s, 5)$
- And we continue like this until we reach the desired vocabulary size.
- We also add the special tokens used by the model at the end of this vocabulary. In the case of GPT-2, the only special token is `<|endoftext|>`.

Summary

- "Training" (i.e. building) a tokenizer is not the same as training a Machine Learning model!
- **"Training" a tokenizer** is a statistical process that identifies the best subwords to choose for a given corpus. The exact rules used to choose them depend on the tokenization algorithm. **The process is deterministic**, which means that you always get the same results when you train with the same algorithm on the same corpus.
- **Training a (ML) modèle** uses the gradient descent to further reduce the prediction loss with each iteration. It **is inherently random**.

Next step...



Supplementary Material

Several Tokenizers

Model	BPE	WordPiece	Unigram
Construction	Start with a small vocabulary and learn rules for merging tokens	Start with a small vocabulary and learn rules for merging tokens	Starts with a large vocabulary and learns rules for removing tokens
Construction stage	Merges tokens matching the most frequent pair	Merges tokens corresponding to the most frequent pair, favoring pairs where each individual token is less frequent	Removes all tokens from the vocabulary that will minimize the calculated loss over the entire corpus
Capture	Merging rules and vocabulary	Just a vocabulary	A vocabulary with a score for each token
Encoding	Cut a word into characters and apply the fusions learned during training	Find the longest subword from the beginning that is in the vocabulary and then do the same for the rest of the word	Find the most likely division in tokens, using the scores learned during training
Used by	GPTs, Mistral, Llamas, DeepSeek, RoBERTa, BART, DeBERTa, etc	ALBERT, T5, mBART, BigBird et XLNet	BERT, etc

A website that visualizes tokenization results: <https://tiktokenizer.vercel.app/>

Optimization by Byte-Level BPE

- What we saw in this first course is the algorithm applied at the character level:
 - We start by building a vocabulary directly from the characters of the text.
 - ASCII-encoded characters can be used, which is 256 characters, including numbers, letters, and symbols.
 - One can instead use characters encoded in Unicode, which supports a wider range of characters, is often used in more complex applications to handle various languages and symbols.
- Thus, in all of the above, the algorithm applies to:
 - $(u, g) \rightarrow ug$
 - $(117, 103) \rightarrow 257$

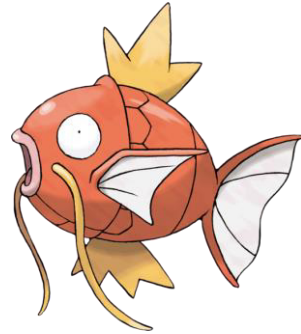
Optimization by Byte-Level BPE

- A simple but very effective idea is not to apply the BPE algorithm directly on characters but to apply it to bytes... We will use the UTF-8 code for the characters.
- Each character is therefore represented on 1 to 4 bytes (for other encodings you can read [1] or [2]).
For example, the word *déjà* is encoded in bytes **100**, 195, 169, **106**, 195, 160
deja would be coded in **100**, 101, **106**, 97
- BPE is then applied at the byte level and not at the character level.

[1] <https://www.w3.org/International/questions/qa-what-is-encoding.fr.html>

[2] <https://www.w3.org/International/articles/definitions-characters/>

SolidGoldMagikarp, the token that derailed language models



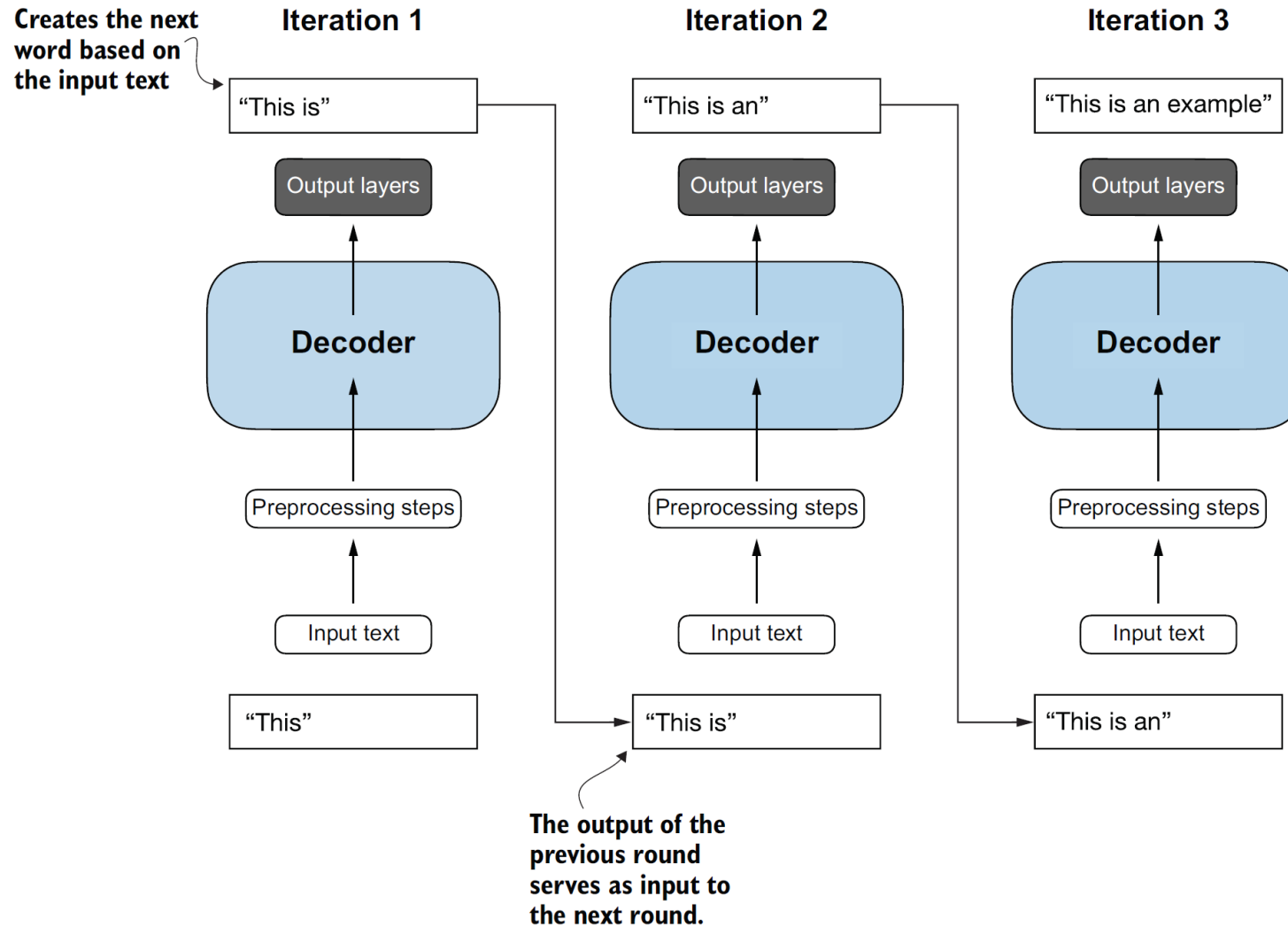
- <https://www.alignmentforum.org/posts/aPeJE8bSo6rAFoLqg/solidgoldmagikarp-plus-prompt-generation>
- <https://www.alignmentforum.org/posts/Ya9LzwEbfaAMY8ABo/solidgoldmagikarp-ii-technical-details-and-more-recent>

Do you have any questions?

Apprentissage auto-supervisé

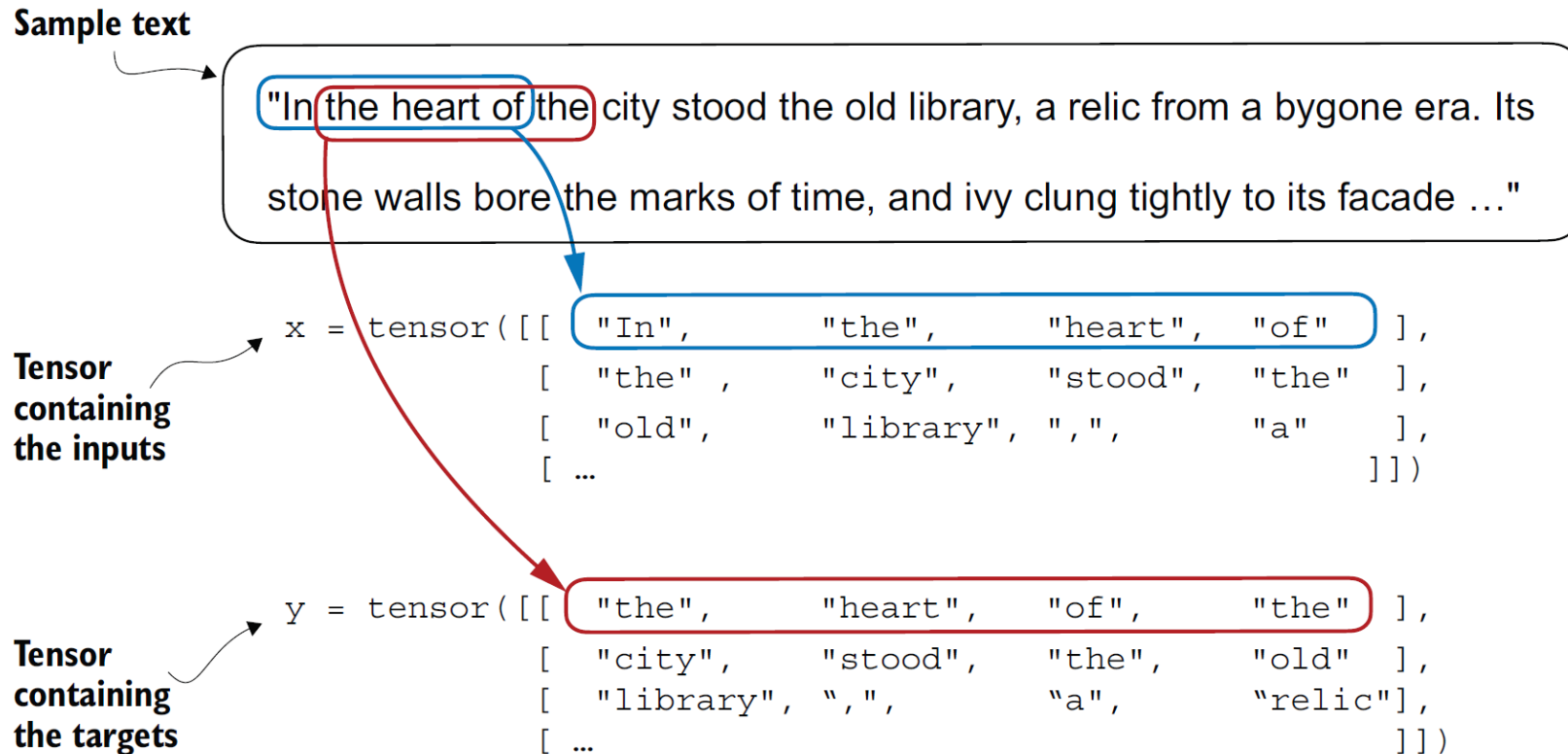
(Self-Supervised Learning)

Ce qu'on veut obtenir à la fin du module



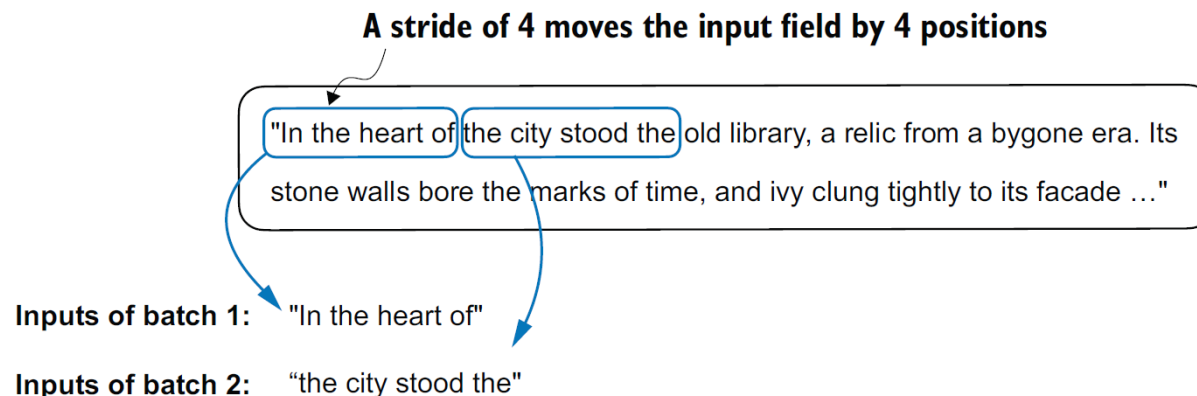
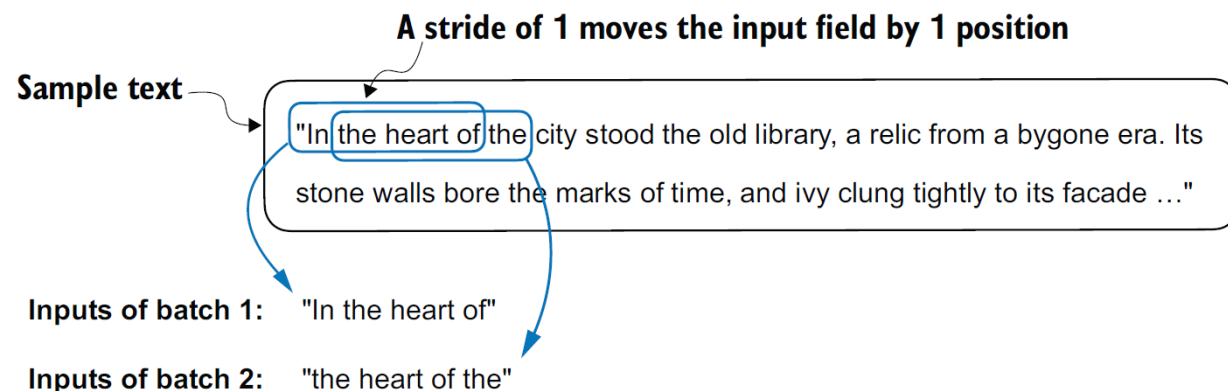
- [illegible]

Préparation d'un training set



Pour mettre en œuvre des *data loader* efficaces, nous collectons les entrées dans un tenseur, x, où chaque ligne représente un contexte d'entrée. Un second tenseur, y, contient les cibles de prédiction correspondantes (mots suivants), qui sont créées en décalant l'entrée d'une position.

Préparation d'un training set



Lors de la création de plusieurs batchs à partir de l'ensemble de données d'entrée, nous utilisons une fenêtre glissante de texte. Si le décalage vaut 1, nous décalons la fenêtre de saisie d'une position lors de la création du batch suivant. Si le décalage est égal à la taille de la fenêtre d'entrée, nous pouvons éviter les chevauchements entre les batchs.